

Akademia Techniczno-Informatyczna w Naukach Stosowanych

Przedmiot	Zajęcia specjalnościowe programowanie
Semestr	Zima 2025 / 2026

Lab 3 06.12.2025

Materiały do ćwiczeń

Proszę przekazywać efekty pracy na Git'a aby ich nie stracić.
W Git'cie, powinny być commity od wszystkich członków grupy a nie tylko od jednego.

Wprowadzenie:

Dysponując strukturą bazy danych (u nas mniej więcej trzy tabele), należy stworzyć obiekty Javowe, które reprezentują tabele i kolumny w bazie danych. Będą to proste obiekty typu POJO. Obiekty te będą "najniżej" w strukturze warstw naszej aplikacji. Obiekty te będą reprezentować warstwę modelu z wzorca MVC. Technika łączenia i odwzorowywania obiektów z/na relacyjną bazę danych nazywa się ORM. Implementacją idei ORM będzie u nas framework **Hibernate** (więcej na następnych zajęciach).

ZADANIE 1

Dodanie brakujących zależności do pom.xml

Przebieg ćwiczenia:

Proszę dodać do pom.xml zależności:

1. Hibernate Core 5.2.10 - podstawowa biblioteka Hibernata to obsługi ORM
2. Hibernate Entity Manager 5.2.10 - dodatkowa biblioteka Hibernata, zawierająca implementację Entitymanagera
3. Spring ORM 4.3.8 - pod-projekt Spring implementujący możliwość używania technologii ORM w Spring
4. Spring WEBMVC 4.3.8 - pod-projekt Spring implementujący możliwość używania wzorca MVC w Springu
5. Wersję proszę przenieść do sekcji <properties></properties>
6. Po dodaniu komenda "mvn clean install" nie może zwracać błędów

```

Wiersz polecenia
C:\Users\cezary\workspace1\Microblog>mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< WMSIS:Microblog >-----
[INFO] Building Microblog 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ Microblog ---
[INFO] Deleting C:\Users\cezary\workspace1\Microblog\target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ Microblog ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 2 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ Microblog ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to C:\Users\cezary\workspace1\Microblog\target\classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ Microblog ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Users\cezary\workspace1\Microblog\src\test\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ Microblog ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to C:\Users\cezary\workspace1\Microblog\target\test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ Microblog ---
[INFO] Surefire report directory: C:\Users\cezary\workspace1\Microblog\target\surefire-reports
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running WMSIS.Microblog.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 sec
[INFO]
[INFO] Results :
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ Microblog ---
[INFO] Building jar: C:\Users\cezary\workspace1\Microblog\target\Microblog-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ Microblog ---
[INFO] Installing C:\Users\cezary\workspace1\Microblog\target\Microblog-0.0.1-SNAPSHOT.jar to C:\Users\cezary\.m2\repository\WMSIS\Microblog\0.0.1-SNAPSHOT\Microblog-0.0.1-SNAPSHOT.jar
[INFO] Installing C:\Users\cezary\workspace1\Microblog\pom.xml to C:\Users\cezary\.m2\repository\WMSIS\Microblog\0.0.1-SNAPSHOT\Microblog-0.0.1-SNAPSHOT.pom
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.248 s
[INFO] Finished at: 2018-08-24T23:31:57+02:00
[INFO]
[INFO]
C:\Users\cezary\workspace1\Microblog>

```

ZADANIE 2

Pliki konfiguracyjne Springa

Aplikacje webowe które używają springa wymagają istnienia dwóch plików z konfiguracją Springa dla naszego projektu. Są to dwa plik typu xml.

Spring w projektach webowych najczęściej jest uruchamiany z użyciem wzorca Front Controller. W uproszczeniu ten wzorzec zakłada posiadanie “nadzorcy”, który zarządza wszystkimi zapytaniami HTTP - tzn. każde zapytanie trafia do tego zarządcy, a następnie on decyduje, jak go obsłużyć (np. do jakiego kontrolera je przekierować). W Springu tą rolę pełni tzw. DispatcherServlet. Servlet to podstawowy element budulcowy dla aplikacji webowych w specyfikacji Java EE. Servlet ten konfigurujemy standardowo w pliku web.xml wskazując konfigurację Springową - plik XML który określa, jak ma się zachowywać Spring.

Plik **web.xml** - Ten plik jest częścią standardu JavaEE i pozwala nam określać m.in. Servlety, Filtry, Listenery itp. My wykorzystamy go jedynie do tego, aby ,oddać’ pełną kontrolę Springowi. Więcej informacji http://docs.oracle.com/cd/E13222_01/wls/docs100/webapp/web_xml.html

Plik **applicationContext.xml** - Plik ten może mieć właściwie dowolną nazwę, ale albo musi być ona podana w pliku web.xml (tak jak my to zrobiliśmy) albo musi mieć nazwę taką jak servlet. Plik ten jest pewnego rodzaju instrukcją, jak ma się zachowywać Spring, co ma udostępniać a czego nie i jak ma

wiązać elementy ze sobą.

Konfiguracja za pomocą **adnotacji** - dzięki włączeniu tego elementu klasy które zdefiniujemy w pliku XML (a nie za pomocą adnotacji) także będą sprawdzane pod kątem posiadanych adnotacji i w razie potrzeby podjęte zostaną odpowiednie kroki.

Component scan - dzięki temu Spring sam “przejrzy” wszystkie dostępne klasy (lub np. zawężone tylko do wybranego pakietu) i te, które są oznaczone za pomocą stereotypów, utworzy w postaci beanów. Dzięki temu nowe np. kontrolery wystarczy napisać i nie musimy się martwić o to, czy pamiętaliśmy o ich konfiguracji i podpięciu.

Przebieg ćwiczenia:

1. Proszę stworzyć w projekcie katalog “src/main/webapp/WEB-INF”
2. Proszę w katalogu stworzyć plik web.xml o takiej treści:

```
web-app id="WebApp_ID" version="2.4"
  xmlns="http://java.sun.com/xml/ns/j2ee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
  <display-name>Microblog</display-name>

  <servlet>
    <servlet-name>mvc-dispatcher</servlet-name>

<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value>/WEB-INF/applicationContext.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
    <servlet-name>mvc-dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>

  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/rootApplicationContext.xml</param-value>
  </context-param>

  <listener>

<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
</web-app>
```

3. Proszę w katalogu stworzyć plik `applicationContext.xml` o takiej treści:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:mvc="http://www.springframework.org/schema/mvc"
        xmlns:context="http://www.springframework.org/schema/context"
        xsi:schemaLocation="
            http://www.springframework.org/schema/mvc
            http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context-4.0.xsd">

    <context:component-scan
        base-package="pl.microblog" /> <!-- Chcemy automatycznie obsługiwać wszystkie
klasy z adnotacjami w tym
        pakiecie. UWAGA!!! każdy projekt może mieć oczywiście inna strukturę pakietów
-->

    <context:annotation-config /> <!-- To na przyszłość, pozwoli nam korzystać z adnotacji
także w klasach,
        które byśmy skonfigurowali ręcznie ->
</beans>
```

4. Plik `rootApplicationContext.xml` - Podobnie jak powyższy, plik ten może mieć właściwie dowolną nazwę, ale albo musi być ona podana w pliku `web.xml` (tak jak my to zrobiliśmy). Różnica pomiędzy tym plikiem, a `applicationContext.xml` jest taka, że teoretycznie możemy mieć kilka aplikacji Springa działających w ramach jednej aplikacji webowej (definiując kilka różnych Servletów w pliku `web.xml`) – ta konfiguracja (uruchamiana przez `ContextLoaderListener`, zdefiniowany zaraz poniżej w pliku `web.xml`) pozwala na współdzielenie pewnych elementów. Proszę w katalogu stworzyć plik `rootApplicationContext.xml` o takiej treści

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation=" http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans-4.0.xsd">
</beans>
```

5. **WAŻNE!!!** Pisząc w dalszej części o “pliku konfiguracyjnym Springa” mamy na myśli `applicationContext.xml`

DataSource to nic innego jak właśnie “źródło danych”. Różnica pomiędzy DataSource (nazywanego też DS) a bazą danych jest taka, że DS może zawierać też elementy związane z optymalizacją zapytań, pulę połączeń itp. Pracując z DS korzystamy z Statements – czyli pojedynczych zapytań. W naszym projekcie DS będzie wskazywało na bazę HSQLDB. DS może wskazywać na dowolne źródło danych - nie musi to być baza danych.

Dodanie DataSource do projektu Spring jest proste, wystarczy dodać beana, który implementuje interfejs `javax.sql.DataSource`. W tym przykładzie użyjemy klasy `org.springframework.jdbc.datasource.DriverManagerDataSource`. Mając skonfigurowaną bazę MySQL (UWAGA!!! u nas jest baza HSQLDB) dodajemy poniższą deklarację do naszego pliku XML:

Przebieg ćwiczenia:

1. Dołącz do pliku konfiguracyjnego Springa dodaj:

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="com.mysql.jdbc.Driver" />
    <property name="url" value="jdbc:mysql://localhost:3306/microblog" />
    <property name="username" value="login" />
    <property name="password" value="tajne" />
</bean>
```

2. Powyżej jest przykład nie związany z projektem MicroBlog. Uzupełnij pola prawdziwymi danymi związanymi z Twoją bazą HSQLDB

ZADANIE 4

Konfiguracja EntityManagera

EntityManager to standardowy sposób wykonywania operacji na obiektach w standardzie JPA (więcej o JPA na następnych zajęciach). Skonfigurujemy fabrykę (czyli sposób, w jaki będą automatycznie tworzone EntityManagery), dzięki czemu będziemy mogli używać EntityManagerów w naszych klasach. Należy pamiętać o tym, że EntityManager to nasz “most” pomiędzy bazą danych a naszą aplikacją.

Konfiguracja EntityManagera w pliku konfiguracyjnym Springa:

```
<tx:annotation-driven />
<bean id="entityManagerFactoryBean"
class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
    <property name="dataSource" ref="dataSource" />
    <property name="packagesToScan" value="pl.wsis.microblog.model" />
    <property name="jpaVendorAdapter">
        <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter">
```

```

        <property name="showSql" value="false" />
        <property name="databasePlatform" value="org.hibernate.dialect.MySQLDialect" />
    </bean>
</property>
<property name="jpaProperties">
    <props>
        <prop key="hibernate.hbm2ddl.auto">update</prop>
    </props>
</property>
</bean>
<bean class="org.springframework.orm.jpa.support.PersistenceAnnotationBeanPostProcessor" />
<bean class="org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor" />
<bean id="transactionManager" class="org.springframework.orm.jpa.JpaTransactionManager" >
    <property name="entityManagerFactory" ref="entityManagerFactoryBean" />
</bean>

```

Najważniejszy jest pierwszy bean, który określa nam zachowanie EntityManagera. Pozostałe beany są “wspierające”, tzn. pełnią określone funkcje (np. zwracają sensowne wyjątki w niektórych przypadkach czy zarządzają tworzeniem i zamykaniem transakcji), ale nie wymagają dodatkowej konfiguracji.

Jeśli chodzi o pierwszy bean jest to implementacja wzorca fabryki (słowo “factory” w nazwie jest nieprzypadkowe) – czyli klasa, która posiada metodę pozwalającą w automatyczny sposób tworzyć instancje obiektów. To pobieranie elementu odbywa się automatycznie bez naszego udziału za pomocą adnotacji . Poszczególne parametry są następujące:

- dataSource – to odniesienie do beana, który definiuje nasz DataSource (UWAGA!!! dodawaliśmy w zadaniu 3)
- packagesToScan – jest to początek nazw pakietów, w których mamy swoje encje (UWAGA!!! każdy może mieć klasy w innych pakiecie, powyżej jest tylko przykład)
- jpaVendorAdapter – to z kolei bean (w naszym wypadku, możemy też zdefiniować go oddzielnie i użyć tylko atrybutu ref żeby wskazać jego nazwę). Bean ten ma dwa parametry dodatkowe:
 - showSQL – steruje tym, czy na konsole mają być wypisywane zapytania SQL, które trafiają do bazy danych. Jest to przydatne przy rozwijaniu aplikacji (możemy łatwo zobaczyć, czy operacja, która powinna się wykonać, faktycznie została wykonana)
 - databasePlatform – wskazuje nazwę klasy, która opisuje naszą bazę danych. Ponieważ różne bazy danych (np. MySQL, PostgreSQL, Oracle) w różny sposób rozszerzają standardową składnię języka SQL, Hibernate stara się wykorzystywać te różnice, żeby zoptymalizować czy przyspieszyć pewne rzeczy. W tym miejscu możemy więc wskazać z jakiej bazy danych korzystamy, aby ‘pomóc’ bibliotece Hibernate zoptymalizować swoje działania
- jpaProperties – tutaj możemy zdefiniować wszystkie niestandardowe ustawienia, np. specyficzne dla naszej implementacji, w naszym przypadku wykorzystamy przede wszystkim jedną opcję:
 - hibernate.hbm2ddl.auto – określa ona, jak hibernate ma się zachowywać przy uruchomieniu. Dostępne jest kilka opcji:
 - validate – tylko weryfikuje, tej opcji powinniśmy używać w działającej aplikacji, jeśli pojawi się jakaś niespójność, aplikacja się nie uruchomi
 - update – Hibernate w przypadku natrafienia na niespójność spróbuje ją usunąć poprzez modyfikację schematu; narażamy się przez to na utratę danych, opcja nadaje się głównie do testowania i rozwoju
 - create – tworzy schemat i strukturę, usuwając istniejące dane
 - create-drop – podobnie jak create tworzy schemat i strukturę usuwając istniejące dane,

ale przy zamykaniu aplikacji automatycznie usuwa całą zawartość używanej bazy danych

Przebieg ćwiczenia:

1. Proszę dodać do pliku konfiguracyjnego Springa

```
<tx:annotation-driven />
<bean id="entityManagerFactoryBean"
  class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
  <property name="dataSource" ref="dataSource" />
  <property name="packagesToScan" value="pl.wwsis.microblog.model" />
  <property name="jpaVendorAdapter">
    <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter">
      <property name="showSql" value="false" />
      <property name="databasePlatform" value="org.hibernate.dialect.MySQLDialect" />
    </bean>
  </property>
  <property name="jpaProperties">
    <props>
      <prop key="hibernate.hbm2ddl.auto">update</prop>
    </props>
  </property>
</bean>
<bean class="org.springframework.orm.jpa.support.PersistenceAnnotationBeanPostProcessor" />
<bean class="org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor" />
<bean id="transactionManager" class="org.springframework.orm.jpa.JpaTransactionManager" >
  <property name="entityManagerFactory" ref="entityManagerFactoryBean" />
</bean>
```

2. Proszę zadbać o ustawienie poprawnych wartości dla swojego projektu! Np. nazwa dataSource, wskazanie pakietów do skanowania z modelem, etc.